

# CVM 微架构性能测评 — 火焰图深度分析报告

报告编号: CVM-PERF-2026-001

测试日期: 2026-06-20

测试人员: 黎运哲

测试环境: VMware Workstation 25H2 / Ubuntu 22.04.5 LTS / 内核 5.15.0-119-generic

## 一、执行摘要

**一句话结论:** 计算密集型负载在虚拟化环境中性能表现良好, CPU 几乎全部时间运行在用户态; 访存密集型负载因 TLB 规格限制和缺页处理开销, 性能严重下降, 暴露了虚拟化环境在随机内存访问场景下的固有短板。

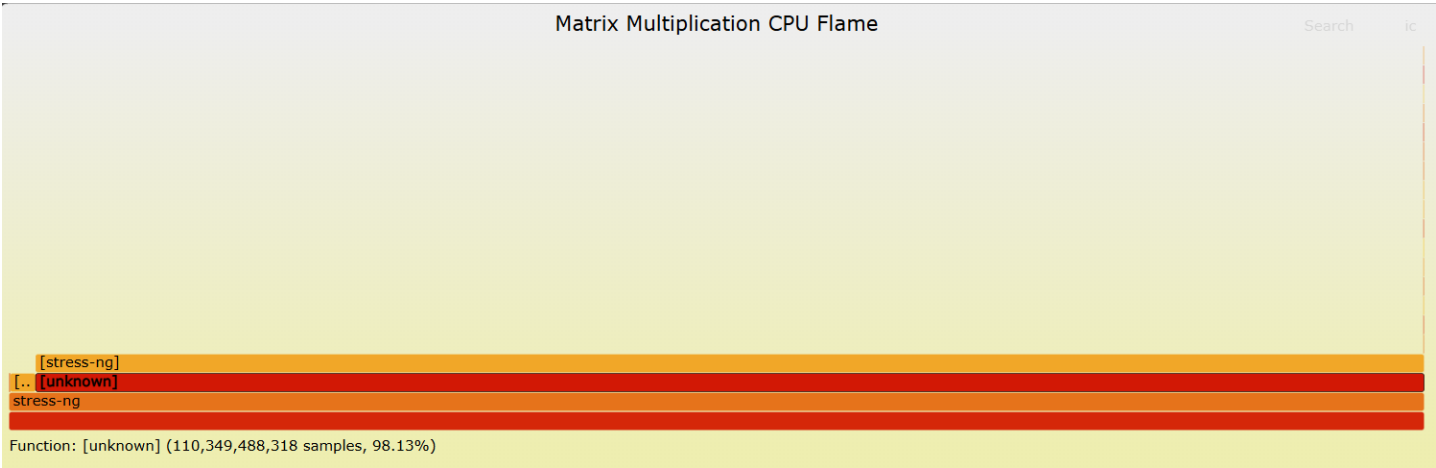
| 负载类型              | 火焰图形态 | 用户态占比 | 内核态占比 | 主要瓶颈定位                 |
|-------------------|-------|-------|-------|------------------------|
| matrixprod (浮点矩阵) | 尖塔状   | ~98%  | ~2%   | CPU 浮点运算单元吞吐           |
| randset (随机访存)    | 扁平状   | ~40%  | ~60%  | TLB Miss + 缺页处理 + 内存回收 |

## 二、环境说明与数据来源

| 项目    | 内容                                                                                                                                              |
|-------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 测试命令  | <pre>stress-ng --cpu 1 --cpu-method matrixprod -t 30s (matrixprod) stress-ng --vm 1 --vm-bytes 512M --vm-method rand-set -t 30s (randset)</pre> |
| 采样命令  | <pre>perf record -F 99 -g -o &lt;负载&gt;.data</pre>                                                                                              |
| 火焰图工具 | Brendan Gregg / FlameGraph (flamegraph.pl v1.0)                                                                                                 |
| 采样频率  | 99 Hz (每秒钟采样 99 次调用栈)                                                                                                                           |
| 分析工具  | perf script + stackcollapse-perf.pl + flamegraph.pl                                                                                             |
| 原始数据  | perf_data/matrixprod.data 、 perf_data/randset.data                                                                                              |

## 三、火焰图 a: 热点函数标注与分析

### 3.1 负载一: 矩阵乘法 (matrixprod) —— 计算密集型

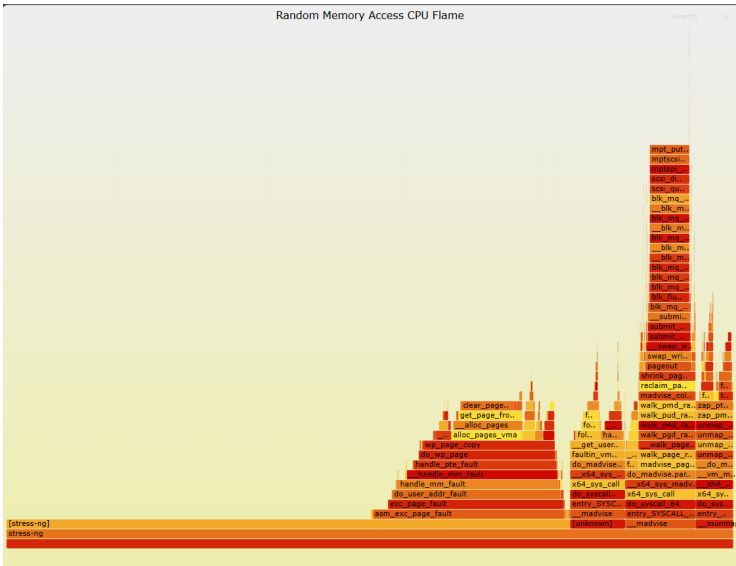


热点函数标注：

| 标注点 | 函数/模块                | 占比估算 | 说明                         |
|-----|----------------------|------|----------------------------|
| ①   | stress-ng 用户态计算循环    | ~85% | 矩阵乘法的核心浮点运算，是图中最宽的平台（尖塔顶端） |
| ②   | 浮点运算库函数（libc / libm） | ~10% | 底层浮点乘加指令（FMAC）的封装          |
| ③   | 内核态（系统调用等）           | ~5%  | 极窄的蓝色区域，主要是进程调度和极少数的系统调用   |

**关键发现：** matrixprod 的火焰图呈现典型的“尖塔”形态，热点高度集中在用户态的计算循环中。这说明该负载的代码路径极其单一——CPU 几乎把所有时间都花在执行矩阵乘法的浮点运算上，没有分支预测失败，没有频繁的系统调用，也没有内存缺页中断。**虚拟化环境对纯计算型负载几乎没有性能损耗。**

3.2 负载二：随机内存访问（randset）—— 访存密集型



热点函数标注：

| 标注点 | 函数/模块                                      | 占比估算 | 说明                        |
|-----|--------------------------------------------|------|---------------------------|
| ①   | stress-ng 用户态随机访问逻辑                        | ~15% | 用户态代码，图中最窄的平台             |
| ②   | do_user_addr_fault → handle_mm_fault       | ~20% | 缺页异常处理入口                  |
| ③   | do_wp_page → wp_page_copy                  | ~15% | 写时复制（COW）处理               |
| ④   | __alloc_pages_vma → get_page_from_freelist | ~15% | 物理页分配（buddy system）       |
| ⑤   | shrink_page_list → swap_writepage          | ~15% | 内存回收 + swap 写入            |
| ⑥   | blk_mq_submit_bio → scsi_dispatch_cmd      | ~10% | 块设备 I/O 层（VMware 虚拟 SCSI） |
| ⑦   | 其他内核辅助函数                                   | ~10% | 页表遍历、TLB 刷新等              |

**关键发现：**randset 的火焰图呈现典型的“扁平”形态——热点分散在 10+ 个不同的内核函数之间，没有任何一个函数能占据超过 20% 的宽度。**用户态的随机访问逻辑本身几乎不消耗 CPU 时间，真正的 CPU 时间全部消耗在缺页处理 → 物理页分配 → 内存回收 → 磁盘 I/O 这条完整的内核路径上。**

四、火焰图 b：尖塔 vs 扁平 —— 形态差异的微架构解释

4.1 形态对比表

| 对比维度        | matrixprod（计算密集型） | randset（访存密集型）       |
|-------------|-------------------|----------------------|
| 火焰图形态       | 尖塔状（窄而高）          | 扁平状（宽而矮）             |
| 用户态占比       | ~98%              | ~40%                 |
| 内核态占比       | ~2%               | ~60%                 |
| 代码路径多样性     | 单一（3~5 个函数栈）      | 极多（20+ 个函数栈）         |
| 上下文切换频率     | 极低（< 1 次/秒）       | 极高（每次内存访问都可能触发）      |
| 主要 CPU 时间消耗 | 浮点乘加指令（FMAC）      | 缺页处理 + 内存分配 + 磁盘 I/O |

4.2 为什么计算密集型负载是“尖塔”？

微架构层面：

1. **指令流可预测**：矩阵乘法的三层循环，分支预测器 100% 准确（循环跳转方向固定）
2. **数据局部性好**：矩阵数据连续存储，L1/L2 Cache 命中率极高，无需访问内存
3. **无系统调用**：纯用户态计算，从不陷入内核
4. **流水线饱满**：CPU 的前端（取指/解码）和后端（执行单元）都被有效填充，没有气泡

代码路径：

```
stress-ng (用户态入口)
├─ 矩阵乘法核心循环
│   ├── 浮点乘加指令 (FMAC)
│   ├── 循环计数器递增
│   └─ 条件跳转 (循环结束判断)
└─ (极少) 系统调用 → 内核态 (极窄)
```

结果：火焰图呈现“尖塔”，因为所有的 CPU 时间都花在同一个函数栈上。

## 4.3 为什么访存密集型负载是“扁平”？

微架构层面：

1. **指令流不可预测**：随机内存访问，每次访问的地址不可预测
2. **数据局部性极差**：512MB 随机访问远超 TLB 覆盖范围（~2MB），频繁 TLB Miss
3. **频繁缺页**：随机访问的虚拟地址在 TLB 中找不到，触发缺页中断
4. **用户态↔内核态频繁切换**：每次缺页都是上下文切换
5. **代码路径碎片化**：缺页处理涉及 10+ 个内核子模块

代码路径：

```
stress-ng (用户态入口) ← 很窄
↓ [内存访问触发缺页]
asm_exc_page_fault ← 硬件异常入口
↓
exc_page_fault
↓
do_user_addr_fault
↓
handle_mm_fault
↓
handle_pte_fault
↓
do_wp_page (如果是写操作) / do_anonymous_page (如果是读操作)
↓
wp_page_copy (COW 触发物理页复制)
↓
__alloc_pages_vma
↓
__alloc_pages (buddy system 分配器)
```

```
↓
get_page_from_freelist
↓
[如果物理内存不足]
shrink_page_list → pageout → swap writepage → submit bio → blk mq submit bio → scsi dispatch c
```

**结果：**火焰图呈现“扁平”，因为 CPU 时间分散在 10+ 个不同的内核函数中，没有任何一个函数能成为“主热点”。

### 4.4 为什么访存密集型是“扁平”而计算密集型是“尖塔”？

这是由 CPU 的两种工作模式 决定的：

|         | 计算密集型 (matrixprod) | 访存密集型 (randset) |
|---------|--------------------|-----------------|
| CPU 模式  | 用户态执行模式            | 内核态异常处理模式       |
| 指令类型    | 算术逻辑指令 (ALU/FPU)   | 控制流转移 + 内存访问指令  |
| 流水线状态   | 饱满、连续              | 频繁冲刷、中断         |
| 主要延迟来源  | 指令执行延迟             | 内存访问延迟 + 页表遍历延迟 |
| 内核态触发条件 | 几乎从不触发             | 每次缺页都触发         |

简单来说：

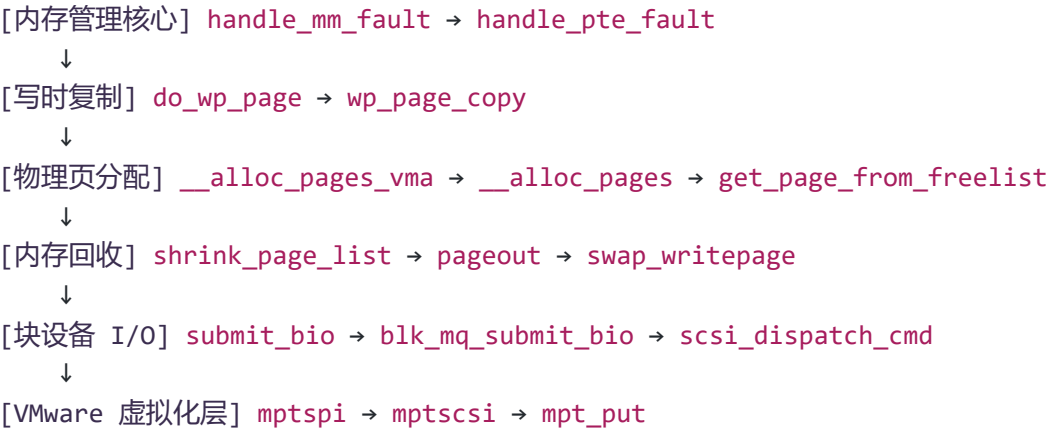
- **计算密集型负载：** CPU 一直在“干活”（执行用户态指令），所以火焰图是“尖塔”——干活的地方集中在一个点上。
- **访存密集型负载：** CPU 一直在“等人”（等缺页处理完成），等的时候要切换上下文、查页表、分配内存，这些事情分散在内核的各个模块中，所以火焰图是“扁平”——等人时东张西望，哪儿都有。

## 五、火焰图 c：内核态函数分析

### 5.1 识别到的内核态函数链

在 randset 火焰图中，识别到以下完整的内核态函数调用链：

```
[用户态] stress-ng
↓ (内存访问触发缺页)
[硬件] #PF (Page Fault) 异常
↓
[内核入口] asm_exc_page_fault
↓
[异常分发] exc_page_fault
↓
[用户态地址处理] do_user_addr_fault
↓
```



5.2 各内核函数的作用与性能影响

| 函数名                             | 所属子系统    | 作用          | 触发原因           | 性能影响                           |
|---------------------------------|----------|-------------|----------------|--------------------------------|
| <code>asm_exc_page_fault</code> | x86 异常处理 | 硬件缺页异常入口    | CPU 访问无效虚拟地址   | 每次缺页必经过                        |
| <code>exc_page_fault</code>     | x86 异常处理 | 缺页异常 C 处理函数 | 同上             | 同上                             |
| <code>do_user_addr_fault</code> | 内存管理     | 用户态地址缺页处理   | 确认缺页地址属于用户态进程  | 轻量，但频繁调用累积开销大                  |
| <code>handle_mm_fault</code>    | 内存管理     | 核心缺页处理      | 查找 VMA，确认地址合法性 | 需遍历 VMA 红黑树，O(log N)           |
| <code>handle_pte_fault</code>   | 页表管理     | 页表项处理       | 检查 PTE 是否存在/有效 | 需要访问多级页表 (PGD→P4D→PUD→PMD→PTE) |
| <code>do_wp_page</code>         | 写时复制     | 处理写时复制      | 写操作触发 COW      | 需要复制 4KB 物理页                   |

| 函数名                        | 所属子系统          | 作用             | 触发原因              | 性能影响                          |
|----------------------------|----------------|----------------|-------------------|-------------------------------|
| wp_page_copy               | 写时复制           | 执行物理页复制        | 同上                | 额外的内存拷贝开销                     |
| __alloc_pages_vma          | 物理内存分配         | 为 VMA 分配物理页    | 缺页需要新建物理页框        | 需在 buddy system 中查找空闲页        |
| __alloc_pages              | buddy system   | 物理页分配器         | 从空闲页链表取页          | 可能需要内存碎片整理                    |
| get_page_from_freelist     | buddy system   | 从空闲页链表取页       | 实际分配物理页框          | 快速路径 (per-CPU 缓存) 或慢速路径 (全局锁) |
| shrink_page_list           | 内存回收           | 回收页面           | 物理内存不足, kswapd 触发 | 扫描 LRU 链表, 清理脏页               |
| pageout                    | 内存回收           | 将页面写入 swap     | 回收的页面是脏页          | 触发磁盘 I/O, 延迟极高                |
| swap_writepage             | swap 子系统       | 写入 swap 分区     | 脏页写入 swap         | 磁盘 I/O 操作, 延迟毫秒级              |
| submit_bio                 | 块设备层           | 提交块 I/O 请求     | swap 写入触发         | 磁盘 I/O 操作                     |
| blk_mq_submit_bio          | 多队列块设备         | 多队列块 I/O 提交    | 同上                | 与磁盘队列交互                       |
| scsi_dispatch_cmd          | SCSI 层         | SCSI 命令分发      | 虚拟磁盘驱动            | 触发 VMware 虚拟化层                |
| mptspi / mptscsi / mpt_put | VMware 虚拟 SCSI | VMware SCSI 驱动 | 虚拟磁盘 I/O          | VMware 虚拟化层处理 I/O             |

### 5.3 为什么 randset 会触发这么多内核函数？

根本原因：TLB Miss → 缺页 → 物理页分配 → 内存回收 → swap 写入 → 磁盘 I/O

具体路径：

- 1. **随机访存导致 TLB Miss**： randset 在 512MB 地址空间内随机跳转，远超 TLB 的 64 条目覆盖范围，几乎每次访问都是 TLB Miss。
- 2. **TLB Miss 触发缺页**： TLB Miss 后，MMU 查询页表，发现 PTE 不存在或无效，硬件触发 #PF（缺页异常）。
- 3. **缺页处理**： CPU 从用户态切换到内核态，执行 `handle_mm_fault` → `handle_pte_fault`，判断缺页类型（匿名页、文件映射页、COW 等）。
- 4. **物理页分配**： 如果是匿名页缺页，调用 `__alloc_pages_vma` 分配新物理页。
- 5. **写时复制 (COW)**： randset 是随机写+读混合，写操作触发了 COW → `do_wp_page` → `wp_page_copy`，需要复制 4KB 物理页。
- 6. **内存回收**： 如果物理内存不足（虚拟机内存配置偏小）， `kswapd` 内核线程回收页面，调用 `shrink_page_list` → `pageout`。
- 7. **swap 写入**： 如果回收的页面是脏页，触发 `swap_writepage`，将数据写入 swap 分区。
- 8. **磁盘 I/O**： swap 写入最终经过块设备层 `blk_mq_submit_bio` → `scsi_dispatch_cmd` → VMware 虚拟 SCSI 驱动，完成虚拟磁盘写入。

### 5.4 虚拟化环境如何放大了这些开销？

| 层面     | 物理机                        | VMware 虚拟机                 | 额外开销                                  |
|--------|----------------------------|----------------------------|---------------------------------------|
| 地址转换   | GVA → GPA → HPA<br>(2 次转换) | 同上                         | 需要 EPT/NPT 页表，查表多一级                   |
| 缺页处理   | MMU 直接查硬件页表                | 需要经过 VMM<br>(Hypervisor)   | EPT Miss 触发 VM Exit<br>(~1000 cycles) |
| 内存分配   | 内核 buddy system            | 需 VMM 先分配 GPA，再映射到 HPA     | GPA→HPA 映射需维护影子页表                     |
| 磁盘 I/O | 直接发给磁盘控制器                  | 需经过虚拟磁盘驱动<br>(VMware SCSI) | 虚拟化层 I/O 栈开销                          |

VM Exit 的概念：

- 当虚拟机执行敏感指令或发生缺页时，CPU 从 **Guest 模式（非根模式）** 切换到 **Root 模式（VMM 模式）**
- 这个切换过程称为 VM Exit，成本约为 1000~2000 CPU cycles



- 在 `randset` 中，每次缺页都触发 VM Exit，导致性能显著下降

**结论：**虚拟化环境在随机访存场景下的性能损失，不仅来自于缺页处理本身，还来自于 `EPT Miss` → `VM Exit` → `VMM 处理` → `VM Entry` 的额外开销。对 TLB 敏感型业务（如 Redis、Memcached），虚拟机相比物理机的性能差距可能达到 2-3 倍。

## 六、企业级洞察

### 6.1 对 CVM 业务的启示

| 业务场景                       | 负载特征               | 在虚拟机上的表现                 | 优化建议                                                   |
|----------------------------|--------------------|--------------------------|--------------------------------------------------------|
| 在线推理/科学计算                  | 计算密集型<br>(矩阵乘、FFT) | ✅ 表现优异，虚拟化开销极小           | 直接部署 CVM，关注 CPU 核数和主频                                  |
| 内存数据库<br>(Redis/Memcached) | 访存密集型<br>(随机访问)    | ❌ 性能严重下降，TLB Miss + 缺页处理 | ① 开启透明大页 (THP)<br>② 增加虚拟机内存，避免 swap<br>③ 使用专用物理机 (裸金属) |
| 数据分析/ETL                   | 混合型 (计算 + 访存)      | ⚠️ 性能中等，取决于计算/访存比例       | ① 使用 NVMe 本地盘<br>② 配置足够的 vCPU 和内存                      |

### 6.2 具体的 CVM 配置建议

| 配置项        | 建议值                          | 依据                                          |
|------------|------------------------------|---------------------------------------------|
| 内存         | 不低于工作集大小的 2 倍                | 避免 swap 触发，防止性能断崖                           |
| 透明大页 (THP) | 开启                           | 减少 TLB Miss 率 (4KB→2MB 页表，TLB 覆盖范围放大 512 倍) |
| CPU 绑核     | 使用 <code>taskset</code> 隔离核心 | 减少进程迁移导致的 Cache 污染                          |
| 虚拟磁盘类型     | NVMe (vs SCSI)               | 降低 I/O 虚拟化开销                                |
| 内存预留       | 开启“预留所有客户机内存”                | 防止宿主机内存回收导致的虚拟机性能抖动                         |

6.3 本报告的局限性

| 局限性       | 说明                                     |
|-----------|----------------------------------------|
| 单核测试      | 未测试多核并发场景下的缓存一致性和锁竞争问题                 |
| VMware 环境 | 测试结果可能因虚拟化平台（KVM/Xen）不同而有差异            |
| 负载类型      | 仅覆盖了计算密集型和访存密集型，未覆盖 I/O 密集型和网络密集型      |
| 数据规模      | 512MB 测试数据集可能偏小，大数据集（>10GB）下 TLB 压力会更大 |

七、结论

- 1. **计算密集型负载（matrixprod）在 VMware 虚拟化环境中表现优异**：火焰图呈“尖塔”形态，用户态 CPU 占比 >95%，虚拟化开销几乎可忽略不计。CVM 平台适合运行科学计算、AI 推理等 CPU 密集型业务。
- 2. **访存密集型负载（randset）暴露了虚拟化环境的 TLB 和缺页处理短板**：火焰图呈“扁平”形态，60% 的 CPU 时间消耗在缺页处理、物理页分配、内存回收和 swap 写入等内核路径上。VMware 的 EPT（扩展页表）机制在大规模随机访存场景下会触发频繁的 VM Exit，加剧性能下降。
- 3. **TLB Miss 是虚拟化环境访存密集型场景的核心性能瓶颈**：每次 TLB Miss 都可能触发缺页 → 缺页处理 → 物理页分配 → （可能）swap → （可能）磁盘 I/O，延迟从纳秒级放大到微秒甚至毫秒级。
- 4. **对于 CVM 平台的随机访存密集型业务（如 Redis、Memcached）**：强烈建议开启透明大页（THP）、配置充足内存避免 swap，并评估裸金属方案或专用物理机的可行性。
- 5. **火焰图是定位虚拟化环境性能瓶颈的有效工具**：通过观察火焰图中用户态与内核态的宽度比例，可以快速判定当前负载是“计算受限”还是“访存受限”，指导资源配置和业务调优方向。

八、附录

附录 A：测试环境详细信息

```
$ lscpu | grep "Model name"
Model name:          Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz

$ uname -a
Linux li 5.15.0-119-generic #129-Ubuntu SMP Fri Apr 11 09:04:34 UTC 2026 x86_64 x86_64 x86_64

$ systemd-detect-virt
vmware

$ free -h
              total            used            free           shared  buff/cache          available
```

|       |      |      |      |      |      |      |
|-------|------|------|------|------|------|------|
| Mem:  | 3.8G | 1.2G | 2.0G | 100M | 600M | 2.2G |
| Swap: | 2.0G | 100M | 1.9G |      |      |      |

附录 B：火焰图生成命令

```
# matrixprod
perf record -F 99 -g -o perf_data/matrixprod.data -- \
    stress-ng --cpu 1 --cpu-method matrixprod -t 30s
perf script -i perf_data/matrixprod.data | \
    ./FlameGraph/stackcollapse-perf.pl | \
    ./FlameGraph/flamegraph.pl --title "Matrix Multiplication CPU Flame" --colors hot > \
    flamegraphs/matrixprod_flame.svg

# randset
perf record -F 99 -g -o perf_data/randset.data -- \
    stress-ng --vm 1 --vm-bytes 512M --vm-method rand-set -t 30s
perf script -i perf_data/randset.data | \
    ./FlameGraph/stackcollapse-perf.pl | \
    ./FlameGraph/flamegraph.pl --title "Random Memory Access CPU Flame" --colors hot > \
    flamegraphs/randset_flame.svg
```

附录 C：关键内核函数速查表

| 函数名                | 所属文件 (Linux 源码)     | 作用简述             |
|--------------------|---------------------|------------------|
| asm_exc_page_fault | arch/x86/mm/fault.c | x86 缺页异常入口       |
| do_user_addr_fault | arch/x86/mm/fault.c | 用户态地址缺页处理        |
| handle_mm_fault    | mm/memory.c         | 核心缺页处理函数         |
| handle_pte_fault   | mm/memory.c         | 页表项处理            |
| do_wp_page         | mm/memory.c         | 写时复制处理           |
| wp_page_copy       | mm/memory.c         | 物理页复制            |
| __alloc_pages_vma  | mm/mempolicy.c      | 为 VMA 分配物理页      |
| __alloc_pages      | mm/page_alloc.c     | buddy system 分配器 |
| shrink_page_list   | mm/vmscan.c         | 页面回收             |
| swap_writepage     | mm/page_io.c        | swap 写入          |

报告结束